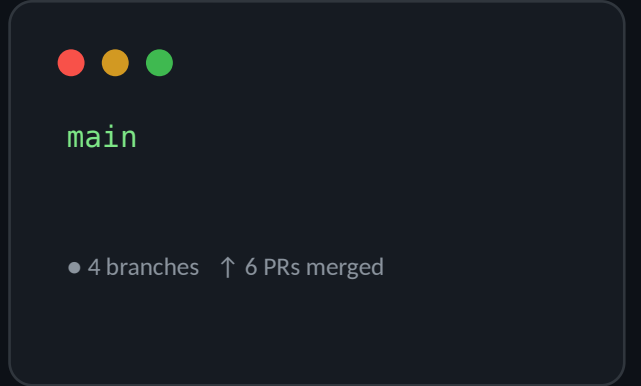




GitHub Repository & Collaboration Setup

Criterion 6 · Phase 2: Environment & GitHub · 10 Marks

SkillsTrack Learner Support Portal — JavaScript Programmer (NSSA TPC)



Evidence walkthrough: README · .gitignore · Issues · Branch workflow · Pull requests · CI linting check

What This Criterion Wants

A repository that proves real, ongoing collaboration — not a single upload.



README

Explains what the project is, the tech stack, and how to run it.



.gitignore

Keeps secrets, .env values and node_modules out of version control.



Issues, assigned

Every rubric criterion tracked as an issue, assigned to a person.



Branch workflow

Feature branches per contributor — nobody commits straight to main.



Pull requests

Changes reviewed and approved before they reach main.



Basic CI check

An automated linting workflow that runs on every push and PR.

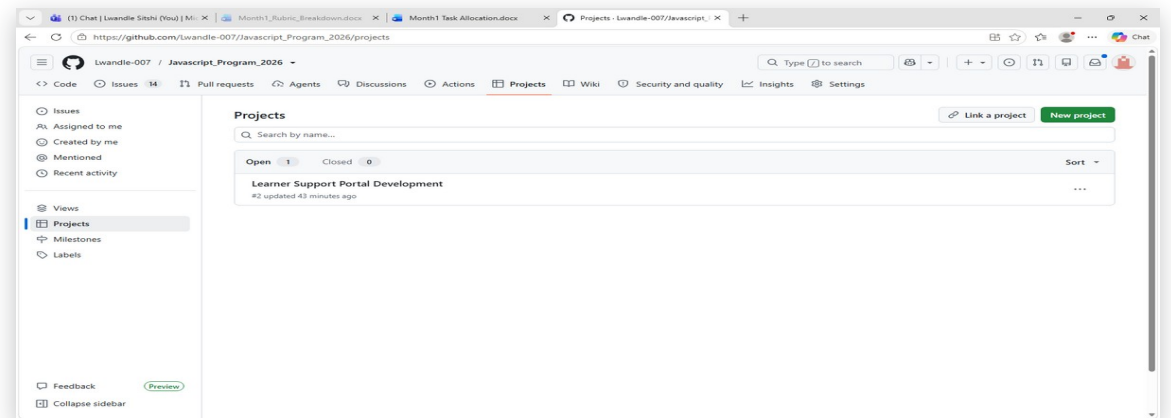
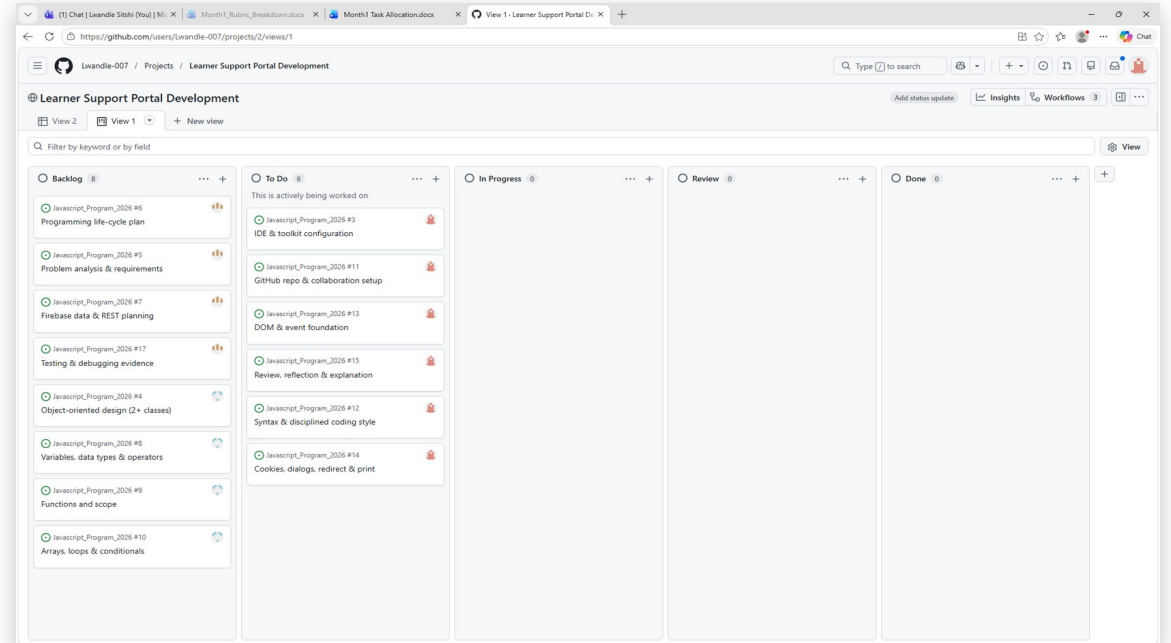
Every Criterion, Tracked

Repository & Project Board



The 14 practical criteria live as [GitHub issues](#) on a project board.

- Backlog holds work not yet started (8 items)
- To Do is what's actively being worked on right now
- Review and Done confirm progress as it happens
- Gives an assessor a live, dated history of the build



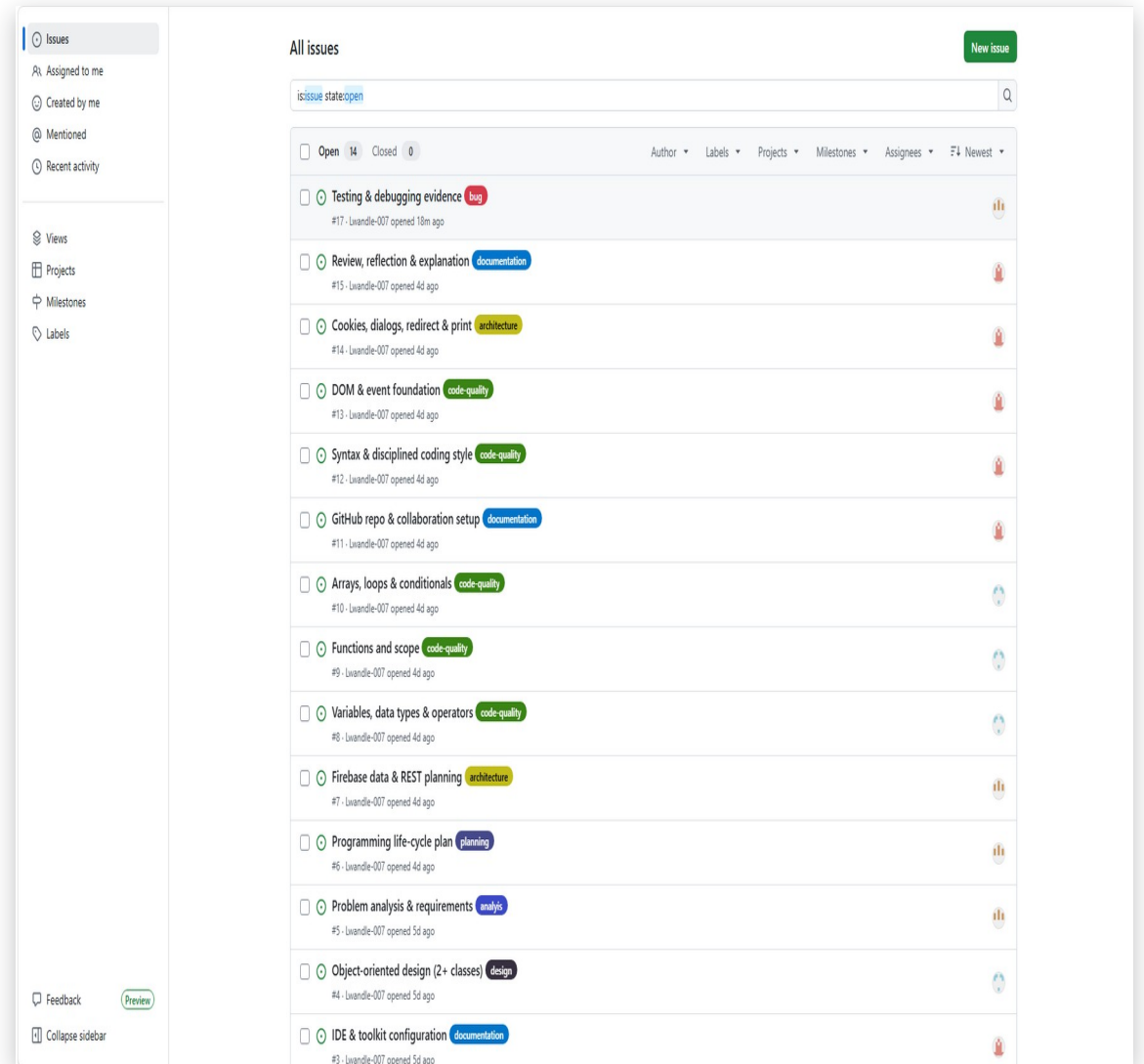
Issues, Labelled & Assigned

Repository Issues



14 open issues — one per rubric criterion.

- Each issue names the exact criterion it satisfies (e.g. “GitHub repo & collaboration setup”)
- Category labels — planning, architecture, code-quality, documentation — group related work
- Assignee avatars show who owns each piece of the build
- A closed issue becomes traceable evidence for the assessor



Evidence: All Issues — 14 open, 0 closed, each labelled and tracked

Criterion 6 • 10 Marks

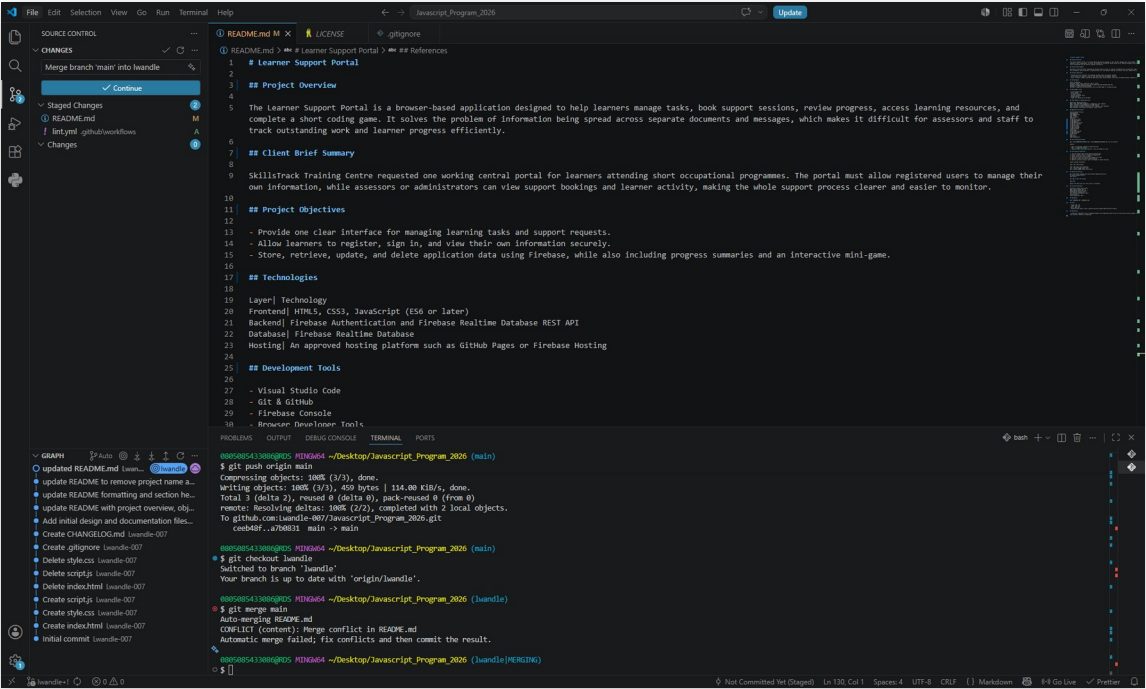
README.md

Documenting the Project



Anyone opening the repo understands the project in 30 seconds.

- Project overview: What the portal is and what problem it solves
- Client brief summary: SkillsTrack's request, in the team's own words
- Objectives: One interface, secure sign-in, full Firebase CRUD
- Technology table: Frontend, backend, database and hosting layers
- Dev tools: VS Code, Git & GitHub, Firebase Console, DevTools



README content, with the pending merge from a teammate's branch visible in Source Control.

.gitignore

Protecting Secrets & Clutter



Nothing sensitive or generated ever reaches the repo.

- `.env` and `.env.*` keep Firebase keys and secrets out of GitHub entirely
- `node_modules/` is excluded — dependencies are reproducible, not stored
- Build and cache folders (`dist`, `.cache`, `.parcel-cache`) stay out too
- Reviewed line-by-line so a real secret is never committed by accident

```
55 .gitignore
56 # Optional stylelint cache
57 .stylelintcache
58
59 # Optional REPL history
60 .node_repl_history
61
62 # Output of 'npm pack'
63 *.tgz
64
65 # Yarn Integrity file
66 .yarn-integrity
67
68 # dotenv environment variable files
69 .env
70 .env.*
71 !.env.example
72
73 # parcel-bundler cache (https://parceljs.org/)
74 .cache
75 .parcel-cache
76
77 # Next.js build output
78 .next
79 out
80
81 # Next.js build / generate output
82 .next
83 dist
84 .output
85
86 # Gatsby files
87 .cache/
88 # Comment in the public line in if your project uses Gatsby and not Next.js
89 # https://nextjs.org/blog/next-9-1public-directory-support
90 # public
91
92 # vuepress build output
93 .vuepress/dist
94
95 # vuepress v2.x temp directory
96 .temp
97
98 # Sveltekit cache directory
99 .svelte-kit/
100
101 # vitepress build output
102 **/.vitepress/dist
103
104 # vitepress cache directory
105 **/.vitepress/cache
106
107 # Docusaurus cache and generated files
108 .docusaurus
109
```

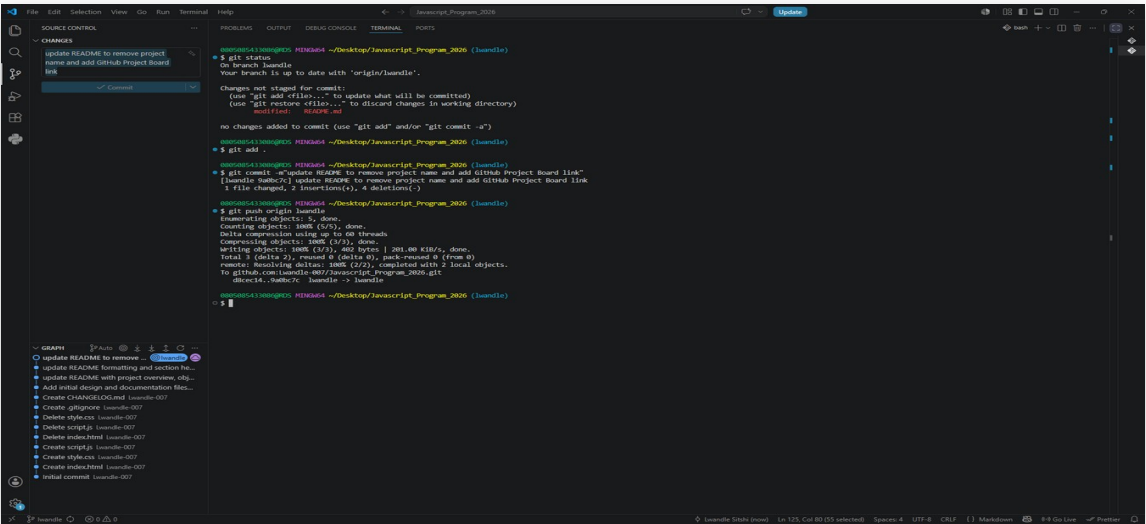
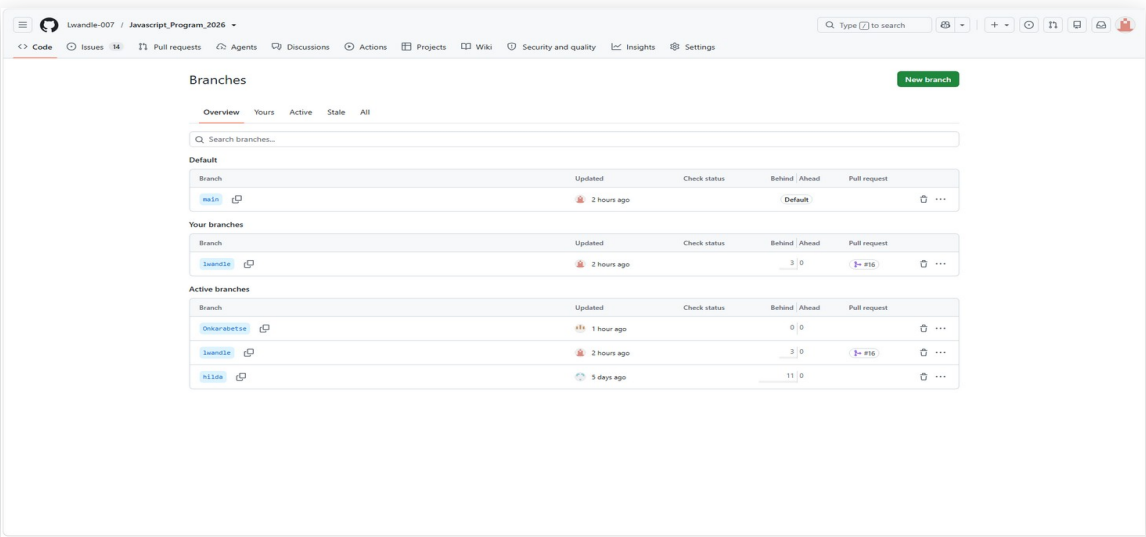
A Real Branch Workflow

Nobody Commits Straight to Main



main stays protected — all work happens on feature branches.

- Three active branches beyond main: lwandle, Onkarabetse, hilda
- Each contributor commits and pushes to their own named branch
- "lwandle" is 3 commits ahead of main, ready for review
- Local work is pushed with git push origin <branch> — never to main directly



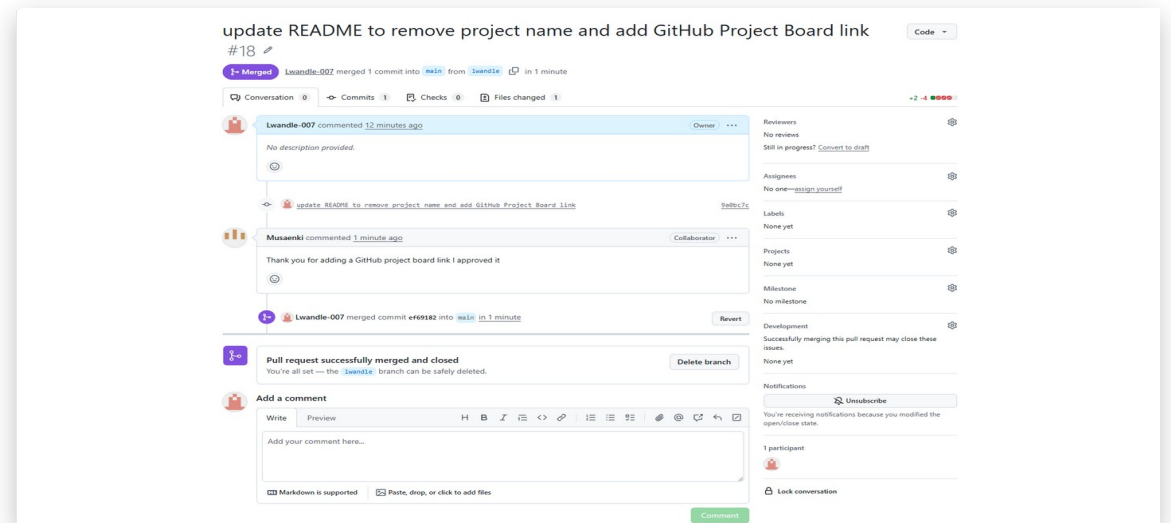
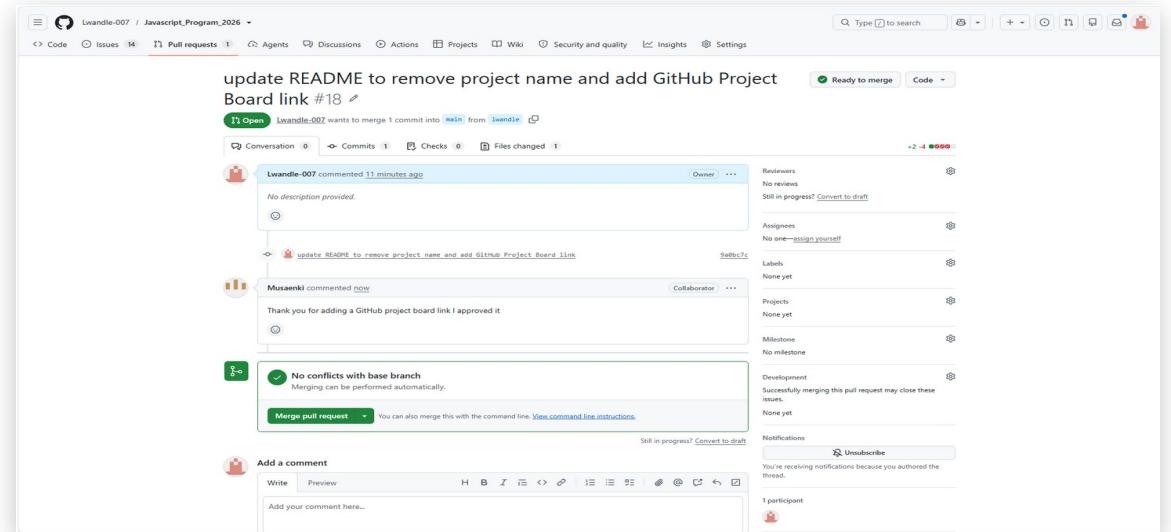
Pull Requests, Reviewed

Review Before Merge



Changes are proposed, checked, and approved — then merged.

- PR #18 opens lwandle → main with a clear title describing the change
- GitHub confirms “No conflicts with base branch” before merging
- A collaborator reviews and comments approval on the change
- Once merged, the source branch is safely deleted to keep things tidy



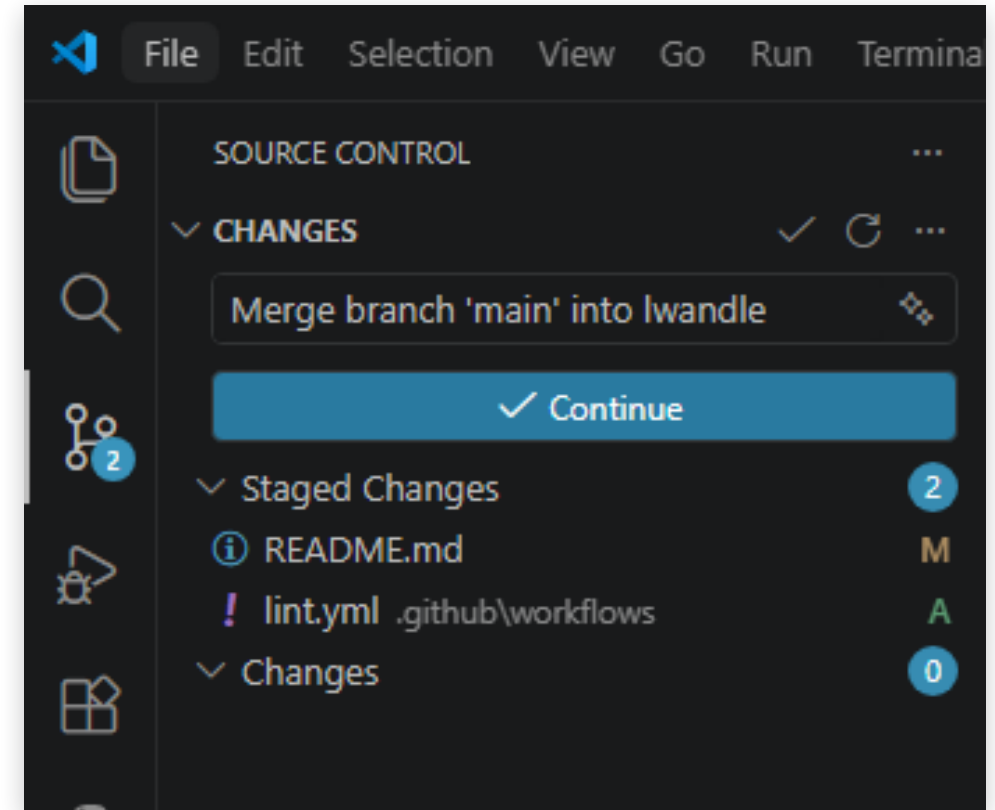
A Basic CI Check

Automated Linting Workflow



Code quality is checked automatically, not just by eye.

- ESLint installed, configured with the eslint:recommended rule set
- Workflow file lives at `.github/workflows/lint.yml`
- Runs automatically on every push and pull request into main
- `lint.yml` appears staged (A) alongside `README.md` in this commit



The linting workflow file, added (A) under `.github/workflows` and staged for commit.

Criterion 6 — Covered

Six requirements, six pieces of evidence.

- ✓ README.md — project overview, brief summary, tech stack
- ✓ .gitignore — secrets and node_modules excluded
- ✓ 14 issues, labelled and assigned to team members
- ✓ Feature branches — main stays protected
- ✓ Pull request opened, reviewed, and merged
- ✓ CI linting workflow committed under .github/workflows

10

out of 10 marks

“Don't leave this to the last minute — it needs a real history of commits over time, not one big dump.”



IDE & Toolkit Configuration

Criterion 5 · Phase 2: Environment & GitHub · 6 Marks

SkillsTrack Learner Support Portal — JavaScript Programmer (NSSA TPC)



PROBLEMS 1

```
ESLint: Identifier already  
declared [Ln 2, Col 5]
```

Evidence walkthrough: Extensions · ESLint setup · Format on save · Linter catch · Formatter · Breakpoint · Framework

What This Criterion Wants

Proof your dev environment actually works — not just that it's installed.



Extensions installed

ESLint and Prettier added to VS Code and active.



Linting catching an error

A real mistake underlined, with the Problems panel showing it.



Formatter before/after

Messy code, then clean — reformatted automatically on save.



Breakpoint working

Execution paused mid-run with the Variables panel visible.



Framework/library installed

package.json or an import line proving a real dependency.



5 screenshots minimum

Every one shows the tool doing its job, not just sitting installed.

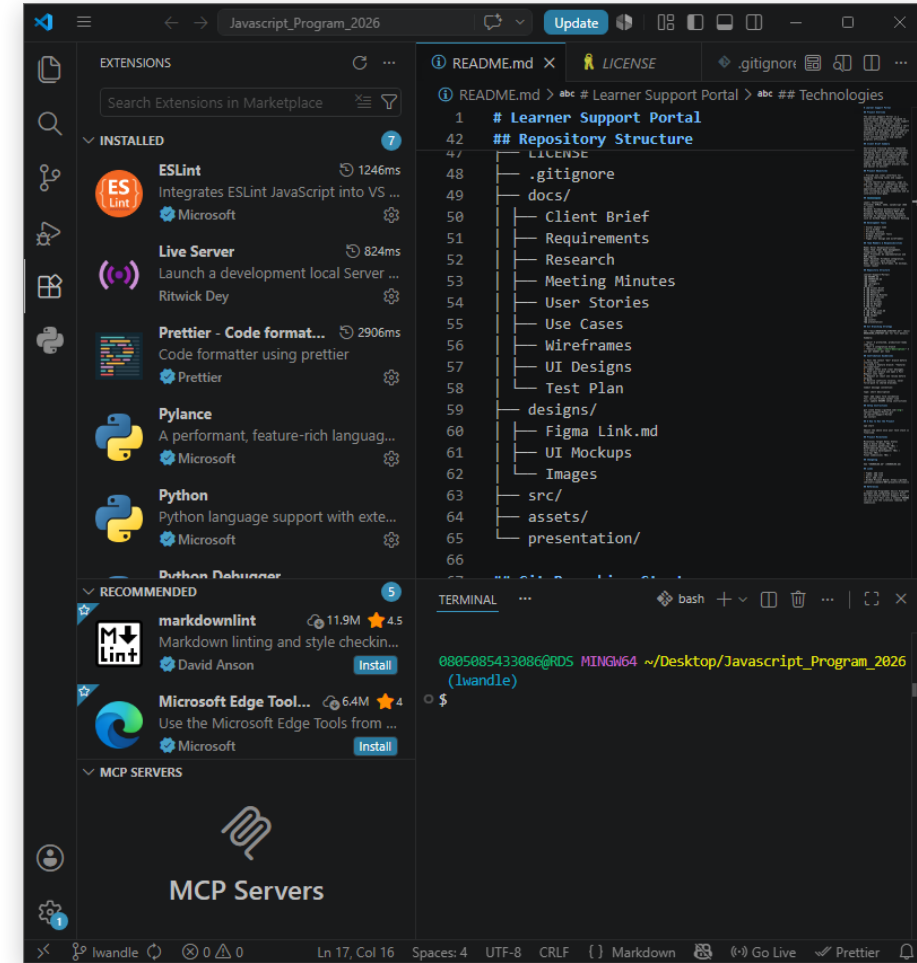
Extensions Installed

Setting Up the Toolkit



VS Code, equipped with the tools the rubric asks for.

- ESLint (by Microsoft) — catches syntax and logic issues live
- Prettier — Code formatter — auto-formats on save
- Both show as Installed in the Extensions panel
- Repository structure also visible — docs, designs, src all organised



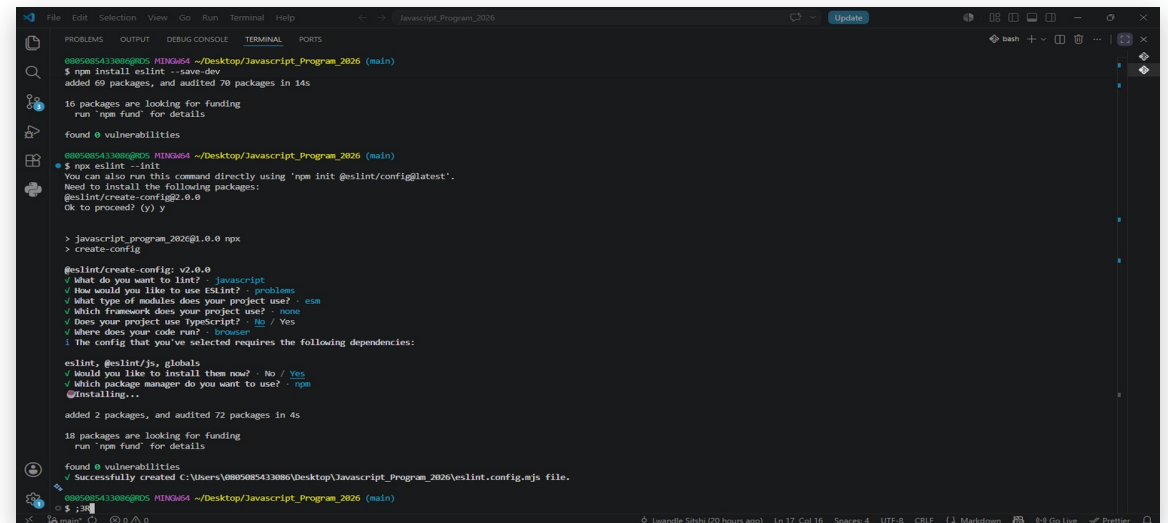
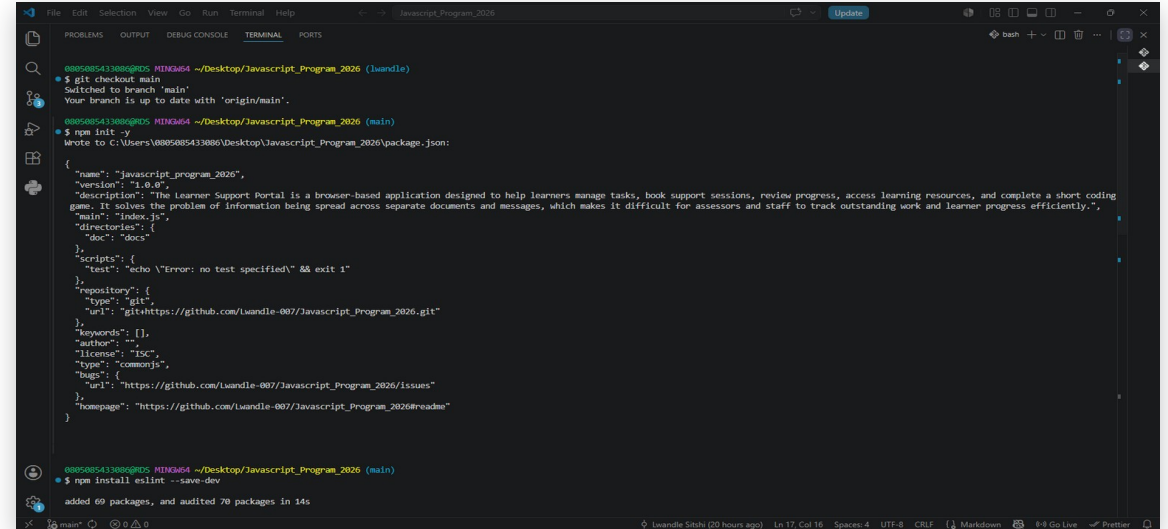
ESLint, Configured

Terminal Setup



The linter isn't just installed — it's wired into the project.

- `npm init -y` creates `package.json` for the project
- `npm install eslint --save-dev` adds ESLint as a dev dependency
- `npx eslint --init` walks through the setup prompts
- Environment: browser · Modules: JavaScript (ESM) · No TypeScript



Format on Save

Editor Settings



Prettier is set as the default formatter, running automatically.

- settings.json sets "editor.formatOnSave": true
- "editor.defaultFormatter": "esbenp.prettier-vscode" points at Prettier
- Every save now reformats the file — no manual step needed
- Other quality-of-life settings configured too (word wrap, autosave)

The screenshot shows the Visual Studio Code editor interface. The main editor window displays the contents of a file named settings.json. The file is a JSON object containing various editor settings. The settings are as follows:

```
1 {
2   "explorer.confirmDelete": false,
3   "terminal.integrated.profiles.windows": {
4     "PowerShell": {
5       "source": "PowerShell",
6       "icon": "terminal-powershell",
7     },
8     "Command Prompt": {
9       "path": [
10        "${env:windir}\\Sysnative\\cmd.exe",
11        "${env:windir}\\System32\\cmd.exe",
12      ],
13      "args": [],
14      "icon": "terminal-cmd",
15    },
16     "Git Bash": {
17       "path": "C:\\Program Files\\Git\\bin\\bash.exe",
18       "args": ["--login", "-i"],
19     },
20   },
21   "terminal.integrated.defaultProfile.windows": "Git Bash",
22   "files.autoSave": "afterDelay",
23   "editor.formatOnSave": true,
24   "editor.wordWrap": "on",
25   "github.copilot.nextEditSuggestions.enabled": true,
26   "github.copilot.enable": {
27     "**": false,
28     "plaintext": false,
29     "markdown": false,
30     "scminput": false,
31   },
32   "editor.defaultFormatter": "esbenp.prettier-vscode",
33 }
34
```

The status bar at the bottom right of the editor shows "Live" and "Prettier", indicating that Prettier is the active formatter.

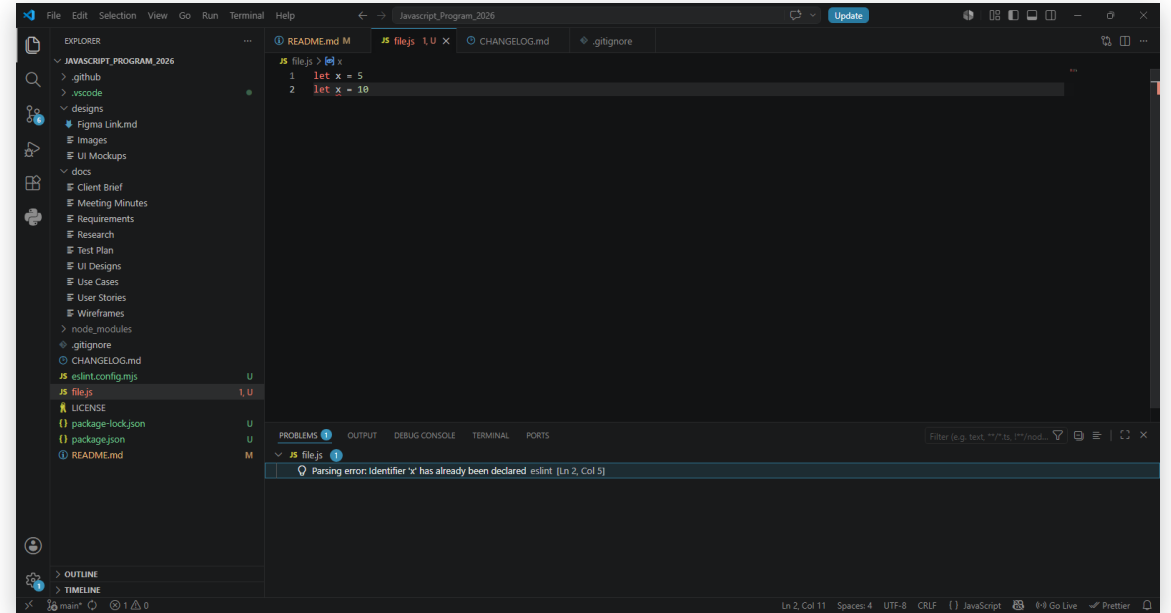
Linter, Catching an Error

Live Error Detection



A real mistake, flagged the moment it's typed.

- `let x = 5` followed by `let x = 10` redeclares the same identifier
- A red squiggly line appears under the second declaration
- The Problems panel spells it out: “Identifier 'x' has already been declared”
- Exact location given — Ln 2, Col 5 — tied to the eslint rule



file.js — duplicate declaration, ESLint parsing error shown in the Problems tab.

Formatter, Before & After

Auto-Reformatting



Deliberately messy code, cleaned up on save — automatically.

- Before: `function add(a,b){return a+b}` — no spacing, no structure
- Save triggers Prettier via format-on-save
- After: proper spacing, braces on their own lines, consistent style
- No manual cleanup — the formatter did all the work

BEFORE

```
JS file.js > add
1 function add(a,b){return a+b}
```

AFTER

```
JS file.js > ...
1 function add(a, b) {
2   return a + b;
3 }
4
```

Formatter ran on save — no manual editing between the two screenshots.

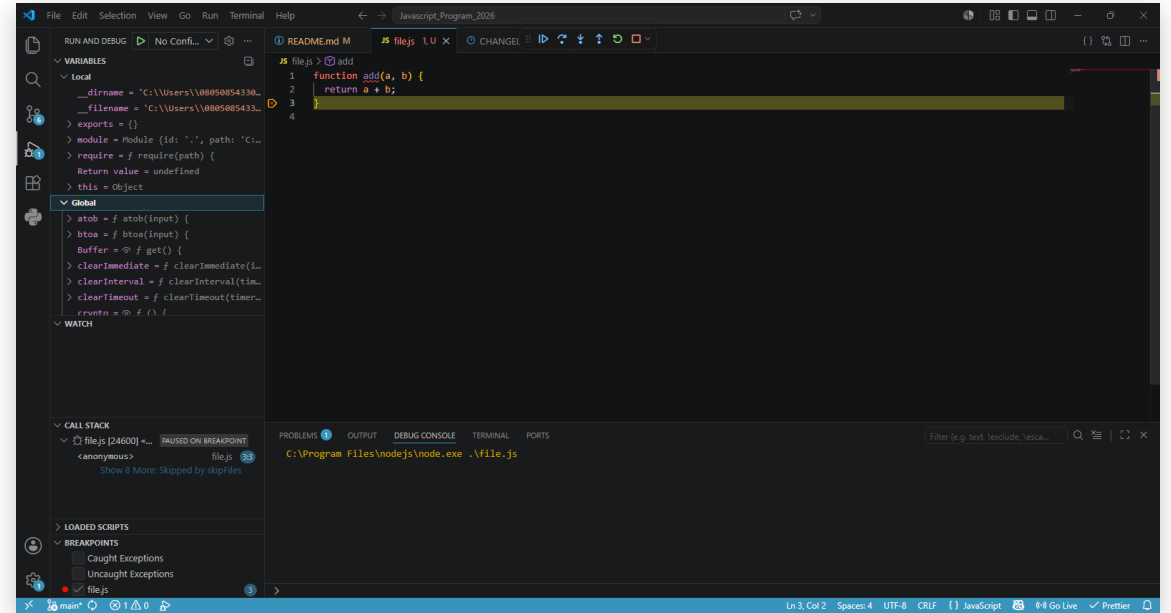
Breakpoint, Working

Debugger in Action



Execution actually pauses — and the state is inspectable.

- A breakpoint set on the closing brace of `add(a, b)`
- F5 launches the Node.js debugger
- Execution pauses exactly there — “PAUSED ON BREAKPOINT” shown
- Variables and Call Stack panels confirm the live program state



Debugger paused on `file.js`, Variables panel open on the left as required.

Framework/Library Installed

Real Dependencies



package.json proves the toolkit is more than a blank folder.

- devDependencies lists eslint, @eslint/js and globals
- Versions pinned — reproducible installs for every teammate
- repository, bugs and homepage fields point at the real GitHub repo
- Proof the toolkit is installed, tracked, and actually used

```
1 {
2   "name": "javascript_program_2026",
3   "version": "1.0.0",
4   "description": "The Learner Support Portal is a browser-based application designed to help learners manage tasks, book support sessions, review progress, access learning resources, and complete a short coding game. It solves the problem of information being spread across separate documents and messages, which makes it difficult for assessors and staff to track outstanding work and learner progress efficiently.",
5   "main": "index.js",
6   "directories": {
7     "doc": "docs"
8   },
9   "scripts": {
10     "test": "echo \"Error: no test specified\" && exit 1"
11   },
12   "repository": {
13     "type": "git",
14     "url": "git+https://github.com/Lwandle-007/Javascript_Program_2026.git"
15   },
16   "keywords": [],
17   "author": "",
18   "license": "ISC",
19   "type": "commonjs",
20   "bugs": {
21     "url": "https://github.com/Lwandle-007/Javascript_Program_2026/issues"
22   },
23   "homepage": "https://github.com/Lwandle-007/Javascript_Program_2026#readme",
24   "devDependencies": {
25     "@eslint/js": "^10.0.1",
26     "eslint": "^10.8.1",
27     "globals": "^17.10.0"
28   }
29 }
```

package.json — devDependencies confirm ESLint is a tracked project dependency.

Criterion 5 — Covered

Five working screenshots, not five installs.

- ✓ ESLint and Prettier extensions installed and active
- ✓ ESLint configured via terminal — `npm init`, `install`, `eslint --init`
- ✓ Format-on-save wired to Prettier in `settings.json`
- ✓ Linter caught a real duplicate-declaration error
- ✓ Formatter cleaned messy code automatically on save
- ✓ Breakpoint paused execution with Variables panel visible
- ✓ `package.json` lists ESLint as a real, tracked dependency

6

out of 6 marks

*"Screenshots proving these tools were only installed
are not enough — every one must show the tool doing
its job."*



Syntax & Disciplined Coding Style

Criterion 7 · Phase 3: Core JavaScript Coding · 6 Marks

SkillsTrack Learner Support Portal — JavaScript Programmer (NSSA TPC)

No single deliverable — this is assessed throughout the whole codebase.



```
$ eslint .
```

```
✓ 0 problems
```

```
clean, consistent, readable
```

What This Criterion Wants

Code that reads like it was written on purpose.



Consistent formatting

Indentation, spacing and style stay the same across every file.



Meaningful names

Variables and functions say what they hold or do — no x, temp, or data2.



Comments where needed

Explain the why behind tricky logic, not the obvious what.



Sensible files & folders

Code organised into logical files, not one giant script.



Small Weight, Big Ripple

Only 6 marks are attached directly — but this criterion colours how every other code-based answer is read.

Every other code-based criterion is read through this lens

Markers form a first impression from formatting alone

Messy code makes markers doubt correct answers too

“Messy code makes markers doubt your other answers too.”

Consistent Formatting

Same Rules, Every File



One style, applied everywhere — automatically.

- Same indentation (2 or 4 spaces) across every file, no mixing
- Consistent spacing around operators, braces and commas
- Prettier + ESLint from Criterion 5 enforce this automatically
- Run `npm run format` and `npm run lint` before every commit

.PRETTIERRC — THE RULES

```
.prettierrc > ...
1 {
2   "singleQuote": false,
3   "semi": true,
4   "tabWidth": 2,
5   "printWidth": 80,
6   "trailingComma": "es5"
7 }
8
```

NPX PRETTIER — CHECK, THEN WRITE

```
PS C:\Users\0809285423086\Desktop\Javascript_Program_2026> npx prettier --check .
Checking formatting...
[warn] .github/workflows/lint.yml
[warn] .vscode/settings.json
[warn] CHANGELOG.md
[warn] eslint.config.mjs
[warn] package-lock.json
[warn] package.json
[warn] README.md
[warn] Code style issues found in 7 files. Run Prettier with --write to fix.
PS C:\Users\0809285423086\Desktop\Javascript_Program_2026> npx prettier --write .
.github/workflows/lint.yml 68ms
.prettierrc 58ms (unchanged)
.vscode/settings.json 3ms
CHANGELOG.md 32ms
designs/Figma Link.md 4ms (unchanged)
eslint.config.mjs 18ms
package-lock.json 50ms
package.json 7ms
README.md 97ms
```

`--check` flags 7 files with style issues; `--write` fixes every one automatically.

Meaningful Names

Say What It Holds or Does



A name should explain itself — no digging through the file to understand it.

- Variables describe the value they hold: `currentDay`, `currentTime`, not `d` or `t`
- Functions describe the action they perform: `selectDay()`, `selectType()`
- `eslint.config.mjs` enforces it: `camelcase`, `no-var` and `prefer-const` are errors
- The linter actually catches slips — `var currentDay` flagged as `no-var`

ESLINT.CONFIG.MJS — THE RULE

```
1 import js from "@eslint/js";
2
3 export default [
4   js.configs.recommended,
5   {
6     rules: {
7       "no-var": "error",
8       "prefer-const": "error",
9       "eqeqeq": "error",
10      "no-unused-vars": "warn",
11      camelcase: ["error", { properties: "always" }],
12    },
13  ],
14 ];
```

CAUGHT ON REAL CODE

Comments Where Needed

Explain the Why, Not the What



A comment earns its place by saving the next reader time.

- JSDoc-style blocks on shared functions: what it does, @param, return
- Inline notes only where logic isn't obvious from the code itself
- Skip comments that just restate the line below them
- A short file header explaining what the file is responsible for

REAL FILE — booking.js

```
1 // booking.js
2 // Handles session-type, day, and time selection, plus the request
3 // summary and submission, for the "Book a Support Session" page.
4
5 var currentTime = "";
6 let currentDay = "";
7
8 /**
9  * Displays the chosen session type in the request summary.
10  * @param (string) value - Label of the selected session type
11  * (e.g. "Progress Review").
12  */
13 function selectType(value) {
14   document.getElementById("summaryType").innerHTML = value;
15 }
16
17 /**
18  * Records the selected day and refreshes the combined day/time summary.
19  * @param (string) value - The selected day (e.g. "Mon 11").
20  */
21 function selectDay(value) {
22   currentDay = value;
23   updateDayTimeSummary();
24 }
25
26 /**
27  * Records the selected time and refreshes the combined day/time summary.
28  * @param (string) value - The selected time slot (e.g. "9:00 AM").
29  */
30 function selectTime(value) {
31   currentTime = value;
32   updateDayTimeSummary();
33 }
34
35 /**
36  * Builds and displays the "day - time" summary text.
37  * Shared by selectDay and selectTime so the string-building logic
38  * only lives in one place.
39  */
```

Three JSDoc blocks in a row — *selectType*, *selectDay*, *selectTime* — each with a purpose and @param.

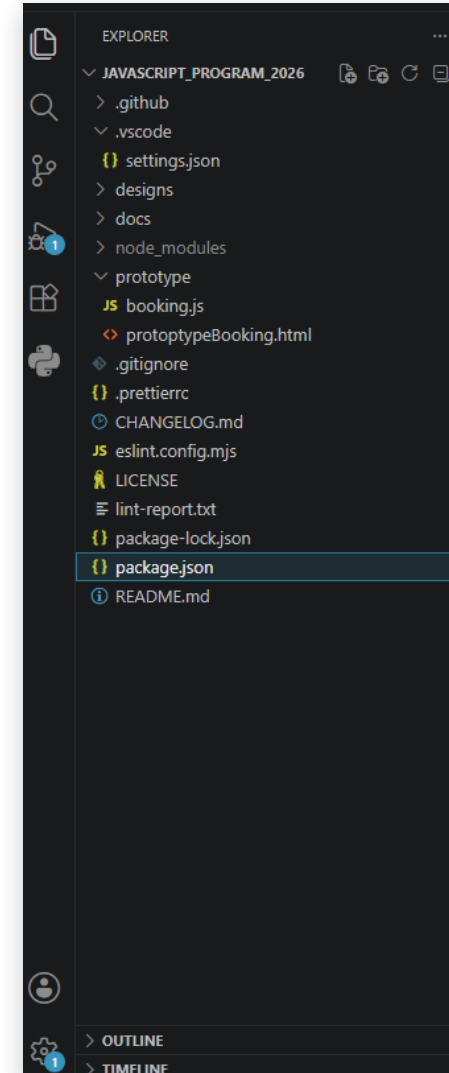
Sensible Files & Folders

Organised, Not Dumped



Every file has an obvious reason to exist where it does.

- Top-level folders separate concerns: docs, designs, prototype
- Config lives at the root: .eslintrc, .prettierrc, package.json
- One feature's logic stays in one file — booking.js owns booking
- No stray scripts loose in the project root



How This Gets Assessed

No Single Deliverable



There's nothing to upload for this one — the whole codebase is the evidence.

- Run `npm run lint` and `npm run format` before every submission
- Markers read every file, not one screenshot — consistency has to hold everywhere
- The ESLint + Prettier setup from Criterion 5 does the enforcing automatically
- A lint report can be piped straight to a file for a written record

`$ npm run lint > lint-report.txt`

```
0805085433086@RDS MINGW64 ~/Desktop/Javascript_Program_2026 (1wandle)
$ npm run lint > lint-report.txt
npm error
npm error Did you mean this?
npm error   npm link # Symlink a package folder
npm error
npm error To see a list of scripts, run:
npm error   npm run
npm error A complete log of this run can be found in: C:\Users\0805085433086\AppData\Local\npm-cache\_logs\2026-08-12T11_09_16_422Z-debug-0.log
0805085433086@RDS MINGW64 ~/Desktop/Javascript_Program_2026 (1wandle)
```

Real terminal output — the lint script piped to a saved report file.

Criterion 7 — Covered

Discipline that shows up in every file, not one screenshot.

- ✓ Formatting stays consistent — enforced by Prettier + ESLint
- ✓ Variables and functions named for what they hold or do
- ✓ Comments explain the why, placed only where they earn it
- ✓ Files and folders organised by feature, not dumped flat
- ✓ Linter and formatter run clean before every submission

6

out of 6 marks

"Messy code makes markers doubt your other answers too."